Technical Report: Multilingual NLI over Swiss Voting Booklets

- **Track:** Track 2A, OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** HackApertus (Online)
- **Team:** OpenAlps, Rohita
- **Repository:** https://github.com/rohIta-k/OpenAlps

1. Summary

Given a claim and a source, the system decides whether the source **entails**, **contradicts**, or is **neutral** toward the claim. The sources are official Swiss voting booklets (long PDFs with several proposals, in German, French and Italian) or short reference passages.

- The hard part is not classification alone, but **finding reliable evidence** in long documents.
- The system extracts PDF text, narrows it to the right proposal, retrieves the best sentences, and asks Apertus v1.5 to classify.
- Every quotation the model returns is **verified against the original PDF** before the prediction is written as JSONL.
- Goal: balance classification quality, evidence reliability and inference cost, while staying reproducible.

2. Architecture

The system is a source-grounded pipeline with five layers:

- **Input handling:** read JSONL cases, validate, remove duplicates.
- **Source processing:** PDF text for Task A; the embedded reference passage for Task B.
- **Evidence retrieval:** BM25 plus multilingual embeddings, combined by Reciprocal Rank Fusion.
- **Inference:** Apertus v1.5 through an OpenAI-compatible API.
- **Validation and output:** parse the answer, verify evidence, write JSONL, run the official evaluator.

PDF processing, retrieval and evidence checks all run locally. Only the classification request goes to the remote model.

	Task A: voting booklet	**Task B: reference text**
Source	Official PDF booklet	Passage supplied in the case
Model	Apertus-v1.5-70B	Apertus-v1.5-8B
Retrieval	Yes (pages, sentences, BM25 + dense)	Not needed
Model output	Label + verbatim evidence quotes	Label only
Final evidence	Verified PDF text with page number	Empty list

3. Use of Apertus

- Apertus v1.5 is the main NLI classifier: the **70B** model for Task A and the **8B** model for Task B.
- Its multilingual ability lets it compare claims and sources written in different languages.
- **Context matters:** we send relevant sentences *with their neighbours*, so qualifications and explanations are not lost.
- **Prompt design:** the three labels are defined explicitly, missing information is neutral (not contradiction), and the response format and length are constrained. This fixed inconsistent labels and malformed outputs seen early on.

4. End-to-End Pipeline

Run with: `uv run python -m src.cli --input data/cases.jsonl --output output/predictions.jsonl`

Step 1: Input and cleaning

- Each line is one JSON case: id, vote, claim, and exactly one source (**booklet** for Task A, **reference** for Task B).
- Malformed records are logged and skipped. Duplicate cases are removed, keeping the first ID.

Step 2: Retrieval (Task A only)

- **Extract text:** PyMuPDF reads each page in top-to-bottom, left-to-right order. Text is cleaned (Unicode normalization, soft hyphens, line-break hyphenation, quotes, whitespace). Physical page numbers are kept.
- **Select pages:** every page is scored against the vote title.
  - An exact title match scores 100; other title words are weighted by how rare they are in the booklet (IDF).
  - Pages scoring 15 or more start a block; neighbours scoring 5 or more extend it; one extra page is added on each side; at most 15 pages are kept.
  - If no page qualifies, the whole booklet is used instead of nothing.
- **Split into sentences:** punctuation-based, with protected abbreviations (Art., z. B., p. ex.). Each sentence keeps its ID, page, section and position.
- **Rank sentences two ways:**
  - **BM25 (lexical):** rewards shared terms, names, dates, numbers and negations.
  - **Multilingual embeddings (semantic):** paraphrase-multilingual-MiniLM-L12-v2 finds sentences with the same meaning, even across languages.
- **Fuse the rankings** with Reciprocal Rank Fusion: score = sum of 1 / (60 + rank) over both rankings. The top 30 sentences are kept and the best 15 are used.
- **Build the context:** add one neighbouring sentence on each side (same page), remove duplicates, restore original order, and label everything with page markers such as [[PAGE 3]] and [S00001 p.3].

Step 3: Classification

- **Prompt:** a system prompt with the decision rules, plus the claim and the retrieved context (Task A) or the full reference passage (Task B).
- **Model call:** temperature 0, JSON-schema output, answered as a label plus evidence.
- **How the model decides:**
  - First it identifies the claim's essential proposition and checks the same vote, proposal and actor.
  - It then compares action, stance, negation, numbers, dates, conditions and scope.
  - **Entailment (0):** the source supports the claim. **Contradiction (2):** a specific source fact conflicts with the claim. **Neutral (1):** anything else (silent, ambiguous or unrelated source).
  - Missing information is never treated as contradiction, and the label is decided *before* picking evidence.
- **Task A evidence:** 1 to 4 verbatim quotes for entailment or contradiction; none for neutral. **Task B:** no evidence.

Step 4: Checking the answer

- The JSON is validated (structure, label, pages, quote length, label/evidence consistency).
- If it is invalid, the pipeline sends **one correction request**. Transient API errors are retried up to six times with backoff.
- **Evidence verification (Task A):** each quote is matched against the original page text (case, spacing, punctuation and hyphenation are ignored) and replaced by the exact PDF span with its 1-based page. If no match is found, fallbacks are tried (hinted page, best word-overlap page, first non-empty page).

- If everything fails, a neutral prediction with empty evidence is written. This is a safety fallback, not a real judgment.

Step 5: Output

- One JSON line per case: id, label (0/1/2), label name, evidence list, and metrics (input tokens, output tokens, inference time).
- Neutral and Task B predictions have empty evidence; non-neutral Task A predictions need at least one evidence item.

5. Data

- **Dataset:** OSTswiss/MNLIoverSwissVotingBooklets v1.1 (MIT License).
- After deduplication: **1,153 unique rows**, giving **2,306 cases** (Task A: claim vs booklet; Task B: claim vs reference text).
- Split by voting day (40/30/30, seed 0) so related cases never cross pools: Development 458, Validation 372, Test 323 rows.
- Only the intended sample data is committed; non-redistributable material stays out of the repository.

6. Evaluation Results

Note: the final results on the entire Hugging Face dataset (all rows) will be updated in the technical report, as running the full dataset is taking time. The numbers below come from the sample runs completed so far.

- **Metric:** macro-F1 over entailment, neutral and contradiction, using the official evaluate.py.
- **Evidence score** (Task A only): share of non-neutral cases where the gold evidence passage was found.
- Token and time figures are averages per case. No invalid outputs were reported in any run.

	Task A baseline	**Task A final**	**Task B baseline**	**Task B final**
Cases	40	60	100	300
Macro-F1	0.8735	0.9658	0.9577	0.9076
Accuracy	0.9000	0.9667	0.9600	0.9100
Evidence score	0.4444 (12/27)	0.5952 (25/42)	n/a	n/a
Input tokens	28,142	3,324	2,065	2,588
Output tokens	108.0	95.3	14.3	14.7
Time (ms)	4,493	3,535	688	579
Passes threshold	Yes (0.6)	Yes (0.6)	Yes (0.7)	Yes (0.7)

What the numbers show

- **Task A improved clearly:** macro-F1 rose from 0.873 to 0.966 and evidence score from 0.444 to 0.595.
- **Task A input tokens fell by about 88%** (28,142 to 3,324), so inference is much cheaper.
- **Task B** scores lower on the larger 300-case set than on the 100-case baseline. The runs use different samples, so they are not a like-for-like comparison.
- The Task A evidence score is lower than its macro-F1: the label can be right even when the retrieved evidence differs from the gold passage.

7. Limitations

- **Retrieval depth:** too few sentences miss qualifications; too many distract the classifier.

- **Proposal selection:** a wrong proposal match can hide the needed evidence; a broad search mixes proposals.
- **Lexical vs semantic:** BM25 misses cross-language paraphrases; embeddings can match a similar topic that is not evidence for the claim.
- **Neutral vs contradiction:** being too strict creates false contradictions; being too cautious misses real conflicts.
- **Evidence checks:** strict quote matching can reject valid quotes after PDF hyphenation or line breaks; loose matching can accept wrong ones.
- **PDF quality:** scanned pages, tables and multi-column layouts can produce missing or misordered text.

8. Reproducibility

The project uses a fixed dataset preparation procedure, a recorded split manifest (docs/splits.json), explicit model settings, and a Docker runtime. To reproduce the results:

- **1. Get the code:** clone the repository and enter the `track_2a/` folder.
- **2. Install dependencies:** run `uv sync`.
- **3. Configure the API:** set the Apertus endpoint and API key as environment variables. Never commit the key.
- **4. Prepare the embedding model:** make sure the multilingual model files are available locally before building the Docker image, because the container runs offline.
- **5. Run:** use `make run` on the default data folder. For custom data in a subfolder, pass its path as the DATA value.
- **6. Validate:** check the predictions against the output contract:

```
uv run python scripts/validate_output.py --cases data/cases.jsonl --predictions
output/predictions.jsonl
```

- **7. Evaluate:** score the predictions with the official evaluate.py from the Hackstarter repository.
- **Tip:** do not use mock mode (MOCK_LLM=1) for results; it returns fake neutral answers and only tests the plumbing.
- More detail: docs/setup.md and docs/configuration.md in the repository.

9. Next Steps

- **Improve proposal selection**
  - Use table-of-contents entries, proposal titles and page ranges to find the right section.
  - Add query expansion from vote metadata so translated or abbreviated titles still match.
  - Goal: stop retrieving evidence from unrelated proposals without dropping the right pages.
- **Analyse Task A errors**
  - Inspect retrieved passages and evidence scores for every wrong prediction.
  - Separate two cases: the evidence was never retrieved, or Apertus saw the right context and still chose the wrong label.
  - Measure retrieval recall against gold evidence before changing any threshold or weight.
- **Refine the classification prompts**
  - Study neutral vs contradiction mistakes: negation, numbers, actor attribution and unsupported claims.
  - Test prompt variants and compare them with the official evaluator, keeping only measured improvements.
- **Optimize retrieval and context**
  - Tune retrieval depth, sentence selection and context length to balance coverage against noise.

  - Try a cross-encoder reranker and contradiction-aware features such as numeric and negation matching.
- **Improve evidence handling**
  - Use source-assigned sentence IDs instead of free-text quotes to reduce quotation and page errors.
  - Make quote matching more robust to PDF artifacts, and log when a fallback was used so evidence quality can be evaluated separately.
- **Complete the full evaluation:** run the entire Hugging Face dataset and update the final results in this report.

License

Released under Creative Commons Attribution 4.0 International (CC BY 4.0), as required by HackApertus. Third-party datasets, models and dependencies keep their own licenses.